

Contents

NurvexThink Website — Foundation Implementation Plan	1
Global Constraints	1
Task 1: Scaffold the Next.js app into the repo	1
Task 2: Configure Prettier, scripts, and a clean lint baseline	2
Task 3: Set up Vitest + Testing Library with a smoke test	3
Task 4: Apply the brand theme (colors, fonts, dark-first) via Tailwind v4	5
Task 5: Install and configure shadcn/ui	6
Task 6: Build the layout shell — Navbar and Footer	7
Task 7: Create placeholder pages for every route	9
Task 8: Wire Supabase clients (browser + server) and env example	10
Task 9: SEO metadata, favicon, and Open Graph from the logo	12
Task 10: GitHub Actions → Vercel deploy pipeline + CI checks	13
Task 11: README and final verification	14
Scripts	15
Docs	15
Self-Review	15

NurvexThink Website — Foundation Implementation Plan

For agentic workers: REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development (recommended) or superpowers:executing-plans to implement this plan task-by-task. Steps use checkbox (- []) syntax for tracking.

Goal: Stand up a deployable, brand-themed Next.js 15 site skeleton — dark/metallic theme, layout shell (navbar + footer), placeholder pages for every route, Supabase client wiring, test tooling, and a GitHub Actions → Vercel deploy pipeline.

Architecture: Single Next.js 15 App Router app (TypeScript) on Vercel, with Supabase as the backend (wired but schema comes in the Backend plan). Server Components by default; client components only at interactive leaves. Styling via Tailwind CSS v4 (CSS-first @theme) + shadcn/ui. This plan produces a site that builds, lints, type-checks, tests green, and deploys — a foundation the later milestones build on.

Tech Stack: Next.js 15, React 19, TypeScript, Tailwind CSS v4, shadcn/ui, @supabase/ssr + @supabase/supabase-js, Vitest + @testing-library/react, ESLint + Prettier, GitHub Actions, Vercel.

Global Constraints

- **Framework:** Next.js 15 (App Router) + React 19. TypeScript everywhere; no any without justification.
- **Backend:** Supabase only (no separate server). Project ref: axbsghyqhhdaiylcksbv; API URL <https://axbsghyqhhdaiylcksbv.supabase.co>
- **Security:** service_role key server-side only — never in client code or the repo. anon key is public-safe. No secrets committed; use .env.local (gitignored) + Vercel/Actions secrets.
- **Brand tokens:** bg #0A0A0B; primary #1E2A44→#2D3A5C; accent #C0C5CE→#E8EBF0. Headings Space Grotesk/Sora; body Inter. Dark-mode-first.
- **Repo/deploy:** public repo nurvexthink/nurvexthink-web; main PR-protected; deploy via GitHub Actions + Vercel token. Conventional Commits (feat:, fix:, chore:, docs:...). Commit identity: Fatima Abdul Raheem <fatima.abdulraheemdev.17@gmail.com>.
- **Perf rules:** Server Components by default; "use client" at leaves; next/image + priority for hero; follow .claude/skills/nurvexthink-nextjs-perf and nurvexthink-3d-perf.

Task 1: Scaffold the Next.js app into the repo

Files: - Create: package.json, tsconfig.json, next.config.ts, postcss.config.mjs, eslint.config.mjs, src/app/layout.tsx, src/app/page.tsx, src/app/globals.css, next-env.d.ts - Note: the repo already

contains CLAUDE.md, docs/, brand/, .claude/, .gitignore, .git/. Scaffold must not clobber these.

Interfaces: - Produces: a working npm run dev/build Next.js app rooted at the repo, with src/ directory and @/* path alias → src/*.

☐ **Step 1: Scaffold with create-next-app into a temp dir, then merge**

Scaffolding directly into a non-empty dir can fail, so scaffold into a temp dir and copy in.

Run:

```
cd "d:/Company"
npx --yes create-next-app@latest .nextscaffold \
  --ts --tailwind --eslint --app --src-dir --import-alias "@/*" \
  --use-npm --no-turbopack --disable-git
```

Expected: completes with "Success! Created .nextscaffold".

☐ **Step 2: Move scaffold files into the repo root (excluding its git/node_modules)**

Run:

```
cd "d:/Company/.nextscaffold"
# copy everything except node_modules and any .git
cp -r package.json tsconfig.json next.config.ts postcss.config.mjs eslint.config.mjs next-env.d.ts p
cp .gitignore "d:/Company/.nextscaffold.gitignore" 2>/dev/null || true
cd "d:/Company" && rm -rf .nextscaffold
```

Expected: src/app/layout.tsx, src/app/page.tsx, src/app/globals.css, package.json now exist at repo root.

☐ **Step 3: Install dependencies**

Run:

```
cd "d:/Company" && npm install
```

Expected: node_modules/ created, no peer-dependency errors that abort install.

☐ **Step 4: Verify dev build compiles**

Run:

```
cd "d:/Company" && npm run build
```

Expected: "Compiled successfully" / route table printed, exit 0.

☐ **Step 5: Commit**

```
cd "d:/Company"
git add -A
git commit -m "chore: scaffold Next.js 15 app (TS, Tailwind, App Router, src dir)"
```

Task 2: Configure Prettier, scripts, and a clean lint baseline

Files: - Create: .prettierrc.json, .prettierignore - Modify: package.json (scripts + devDeps)

Interfaces: - Produces: npm scripts lint, format, typecheck that later tasks and CI call.

☐ **Step 1: Install Prettier + Tailwind plugin**

Run:

```
cd "d:/Company" && npm install -D prettier prettier-plugin-tailwindcss
```

Expected: added to devDependencies.

❑ Step 2: Create .prettierrc.json

```
{
  "semi": true,
  "singleQuote": false,
  "trailingComma": "all",
  "printWidth": 100,
  "plugins": ["prettier-plugin-tailwindcss"]
}
```

❑ Step 3: Create .prettierignore

```
.next
node_modules
public
*.md
docs
brand
```

❑ Step 4: Add scripts to package.json

In the "scripts" block, ensure these exist (merge with existing dev/build/start/lint):

```
{
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start",
    "lint": "next lint",
    "format": "prettier --write .",
    "format:check": "prettier --check .",
    "typecheck": "tsc --noEmit"
  }
}
```

❑ Step 5: Run format + typecheck + lint

Run:

```
cd "d:/Company" && npm run format && npm run typecheck && npm run lint
```

Expected: format rewrites files; typecheck exits 0 with no errors; lint reports no errors.

❑ Step 6: Commit

```
git add -A && git commit -m "chore: add prettier, format, typecheck, and lint scripts"
```

Task 3: Set up Vitest + Testing Library with a smoke test

Files: - Create: vitest.config.ts, vitest.setup.ts, src/lib/utils.ts, src/lib/utils.test.ts - Modify: package.json (test scripts + devDeps)

Interfaces: - Produces: cn(...inputs) class-merge helper used by all components; npm test script used by CI.

❑ Step 1: Install test + util deps

Run:

```
cd "d:/Company" && npm install -D vitest @vitejs/plugin-react jsdom @testing-library/react @testing-library/jest-dom
```

Expected: deps added.

❑ Step 2: Create vitest.config.ts

```
import { defineConfig } from "vitest/config";
import react from "@vitejs/plugin-react";
import path from "node:path";

export default defineConfig({
  plugins: [react()],
  test: {
    environment: "jsdom",
    globals: true,
    setupFiles: ["./vitest.setup.ts"],
  },
  resolve: {
    alias: { "@": path.resolve(__dirname, "./src") },
  },
});
```

❑ Step 3: Create vitest.setup.ts

```
import "@testing-library/jest-dom/vitest";
```

❑ Step 4: Add test scripts to package.json

```
{
  "scripts": {
    "test": "vitest run",
    "test:watch": "vitest"
  }
}
```

❑ Step 5: Write the failing test for cn

Create src/lib/utils.test.ts:

```
import { describe, it, expect } from "vitest";
import { cn } from "@lib/utils";

describe("cn", () => {
  it("merges class names", () => {
    expect(cn("a", "b")).toBe("a b");
  });
  it("dedupes conflicting tailwind classes (last wins)", () => {
    expect(cn("p-2", "p-4")).toBe("p-4");
  });
  it("drops falsey values", () => {
    expect(cn("a", false && "b", undefined, "c")).toBe("a c");
  });
});
```

❑ Step 6: Run test to verify it fails

Run: `cd "d:/Company" && npm test` Expected: FAIL — cannot resolve @/lib/utils (module not found).

❑ Step 7: Implement src/lib/utils.ts

```
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}
```

❑ Step 8: Run test to verify it passes

Run: `cd "d:/Company" && npm test` Expected: PASS — 3 passing.

❑ Step 9: Commit

```
git add -A && git commit -m "test: add vitest setup and cn() utility with tests"
```

Task 4: Apply the brand theme (colors, fonts, dark-first) via Tailwind v4

Files: - Modify: `src/app/globals.css` (theme tokens) - Modify: `src/app/layout.tsx` (fonts + base classes)

Interfaces: - Produces: CSS custom properties / Tailwind theme tokens `--color-bg`, `--color-primary`, `--color-primary-2`, `--color-accent`, `--color-accent-2`; font CSS vars `--font-heading`, `--font-body`.

❑ Step 1: Replace `src/app/globals.css` with brand theme

```
@import "tailwindcss";

@theme {
  --color-bg: #0a0a0b;
  --color-surface: #121319;
  --color-primary: #1e2a44;
  --color-primary-2: #2d3a5c;
  --color-accent: #c0c5ce;
  --color-accent-2: #e8ebf0;
  --color-muted: #8a93a6;

  --font-heading: var(--font-heading), ui-sans-serif, system-ui, sans-serif;
  --font-body: var(--font-body), ui-sans-serif, system-ui, sans-serif;
}

:root {
  color-scheme: dark;
}

body {
  background-color: var(--color-bg);
  color: var(--color-accent-2);
  font-family: var(--font-body);
}

h1, h2, h3, h4 {
  font-family: var(--font-heading);
}
```

❑ Step 2: Wire fonts + base layout in `src/app/layout.tsx`

```
import type { Metadata } from "next";
import { Space_Grotesk, Inter } from "next/font/google";
import "./globals.css";

const heading = Space_Grotesk({
  subsets: ["latin"],
  variable: "--font-heading",
  display: "swap",
});
const body = Inter({
```

```

    subsets: ["latin"],
    variable: "--font-body",
    display: "swap",
  });

export const metadata: Metadata = {
  title: { default: "NurvexThink", template: "%s – NurvexThink" },
  description: "Software, built and published. Custom software on demand.",
};

export default function RootLayout({ children }: { children: React.ReactNode }) {
  return (
    <html lang="en" className={` ${heading.variable} ${body.variable}`}>
      <body className="min-h-dvh bg-bg text-accent-2 antialiased">{children}</body>
    </html>
  );
}

```

☐ Step 3: Verify build + typecheck

Run:

```
cd "d:/Company" && npm run typecheck && npm run build
```

Expected: exit 0, "Compiled successfully".

☐ Step 4: Commit

```
git add -A && git commit -m "feat: apply NurvexThink dark/metallic theme tokens and fonts"
```

Task 5: Install and configure shadcn/ui

Files: - Create: components.json, src/components/ui/button.tsx - Modify: src/app/globals.css (shadcn variables, if added by init)

Interfaces: - Produces: shadcn Button component (and the components.json config) consumed by the layout shell and later pages.

☐ Step 1: Initialize shadcn/ui (non-interactive)

Run:

```
cd "d:/Company" && npx --yes shadcn@latest init -d -b neutral
```

Expected: creates components.json and base CSS variables. -d = defaults, -b neutral = neutral base color (we override with brand tokens already in globals.css).

☐ Step 2: Add the Button component

Run:

```
cd "d:/Company" && npx --yes shadcn@latest add button
```

Expected: creates src/components/ui/button.tsx.

☐ Step 3: Verify build

Run: cd "d:/Company" && npm run build Expected: exit 0.

☐ Step 4: Commit

```
git add -A && git commit -m "chore: set up shadcn/ui with Button component"
```

Task 6: Build the layout shell — Navbar and Footer

Files: - Create: `src/components/layout/navbar.tsx`, `src/components/layout/footer.tsx`, `src/components/layout/`
- Create: `src/lib/nav.ts` (nav link data) - Modify: `src/app/layout.tsx` (render Navbar/Footer around children)

Interfaces: - Consumes: `cn` from `@/lib/utils`; `Button` from `@/components/ui/button`. - Produces:
`NAV_LINKS`: { `href`: string; `label`: string }[] from `@/lib/nav`; `<Navbar />` and `<Footer />` server components.

□ Step 1: Create `src/lib/nav.ts`

```
export const NAV_LINKS: { href: string; label: string }[] = [
  { href: "/products", label: "Products" },
  { href: "/blog", label: "Blog" },
  { href: "/order", label: "Custom Order" },
  { href: "/about", label: "About" },
  { href: "/contact", label: "Contact" },
];
```

□ Step 2: Write the failing Navbar test

Create `src/components/layout/navbar.test.tsx`:

```
import { describe, it, expect } from "vitest";
import { render, screen } from "@testing-library/react";
import { Navbar } from "@components/layout/navbar";

describe("Navbar", () => {
  it("renders the brand wordmark", () => {
    render(<Navbar />);
    expect(screen.getByText("NurvexThink")).toBeInTheDocument();
  });
  it("renders all primary nav links", () => {
    render(<Navbar />);
    for (const label of ["Products", "Blog", "Custom Order", "About", "Contact"]) {
      expect(screen.getByRole("link", { name: label })).toBeInTheDocument();
    }
  });
});
```

□ Step 3: Run test to verify it fails

Run: `cd "d:/Company" && npm test -- navbar` Expected: FAIL — cannot resolve `@/components/layout/navbar`.

□ Step 4: Implement `src/components/layout/navbar.tsx`

```
import Link from "next/link";
import { NAV_LINKS } from "@lib/nav";

export function Navbar() {
  return (
    <header className="sticky top-0 z-50 border-b border-white/10 bg-bg/70 backdrop-blur">
      <nav className="mx-auto flex max-w-6xl items-center justify-between px-6 py-4">
        <Link href="/" className="font-[family-name:var(--font-heading)] text-xl font-bold tracking-
          NurvexThink
        </Link>
        <ul className="hidden items-center gap-8 md:flex">
          {NAV_LINKS.map((link) => (
            <li key={link.href}>
```

```

        <Link
          href={link.href}
          className="text-sm text-muted transition-colors hover:text-accent-2"
        >
          {link.label}
        </Link>
      </li>
    )}
  </ul>
</nav>
</header>
);
}

```

❑ Step 5: Run test to verify it passes

Run: `cd "d:/Company" && npm test -- navbar` Expected: PASS — 2 passing.

❑ Step 6: Implement `src/components/layout/footer.tsx`

```

export function Footer() {
  const year = new Date().getFullYear();
  return (
    <footer className="border-t border-white/10 bg-bg">
      <div className="mx-auto flex max-w-6xl flex-col gap-2 px-6 py-10 text-sm text-muted">
        <p className="font-[family-name:var(--font-heading)] text-accent-2">NurvexThink</p>
        <p>Software, built and published. Custom software on demand.</p>
        <p>© {year} NurvexThink. All rights reserved.</p>
      </div>
    </footer>
  );
}

```

❑ Step 7: Render shell in `src/app/layout.tsx`

Replace the `<body>` contents:

```

<body className={` ${heading.variable} ${body.variable} flex min-h-dvh flex-col bg-bg text-accent-2 `}
  {/* eslint-disable-next-line @typescript-eslint/no-use-before-define */}
  <Navbar />
  <main className="flex-1">{children}</main>
  <Footer />
</body>

```

And add imports at top:

```

import { Navbar } from "@components/layout/navbar";
import { Footer } from "@components/layout/footer";

```

❑ Step 8: Verify build + tests

Run: `cd "d:/Company" && npm test && npm run build` Expected: tests pass, build exits 0.

❑ Step 9: Commit

```

git add -A && git commit -m "feat: add navbar and footer layout shell"

```


Task 7: Create placeholder pages for every route

Files: - Create: `src/app/products/page.tsx`, `src/app/blog/page.tsx`, `src/app/order/page.tsx`, `src/app/about/page.tsx`, `src/app/contact/page.tsx` - Modify: `src/app/page.tsx` (home hero placeholder) - Create: `src/components/page-heading.tsx`

Interfaces: - Consumes: layout shell (already global). - Produces: `<PageHeading title subtitle />` used by all inner pages; routes that return 200.

□ Step 1: Create reusable `src/components/page-heading.tsx`

```
export function PageHeading({ title, subtitle }: { title: string; subtitle?: string }) {
  return (
    <div className="mx-auto max-w-6xl px-6 py-16">
      <h1 className="text-4xl font-bold tracking-tight text-accent-2">{title}</h1>
      {subtitle ? <p className="mt-3 max-w-2xl text-muted">{subtitle}</p> : null}
    </div>
  );
}
```

□ Step 2: Home page `src/app/page.tsx`

```
import Link from "next/link";
import { Button } from "@/components/ui/button";

export default function HomePage() {
  return (
    <section className="mx-auto flex max-w-6xl flex-col items-center px-6 py-28 text-center">
      <h1 className="bg-gradient-to-r from-accent to-accent-2 bg-clip-text text-5xl font-bold tracking-tight">
        NurvexThink
      </h1>
      <p className="mt-6 max-w-2xl text-lg text-muted">
        Software, built and published. Explore our products, read the blog, or request custom
        software on demand.
      </p>
      <div className="mt-10 flex gap-4">
        <Button asChild>
          <Link href="/products">Explore products</Link>
        </Button>
        <Button asChild variant="outline">
          <Link href="/order">Request custom software</Link>
        </Button>
      </div>
      { /* 3D metallic "NT" hero is added in the 3D milestone plan */ }
    </section>
  );
}
```

□ Step 3: Create the five inner pages

`src/app/products/page.tsx`:

```
import type { Metadata } from "next";
import { PageHeading } from "@/components/page-heading";
export const metadata: Metadata = { title: "Products" };
export default function ProductsPage() {
  return <PageHeading title="Products" subtitle="Software we build and publish. Coming soon." />;
}
```

`src/app/blog/page.tsx`:

```
import type { Metadata } from "next";
import { PageHeading } from "@components/page-heading";
export const metadata: Metadata = { title: "Blog" };
export default function BlogPage() {
  return <PageHeading title="Blog" subtitle="Notes on what we build. Coming soon." />;
}
```

src/app/order/page.tsx:

```
import type { Metadata } from "next";
import { PageHeading } from "@components/page-heading";
export const metadata: Metadata = { title: "Custom Order" };
export default function OrderPage() {
  return <PageHeading title="Request custom software" subtitle="Tell us what you need. Form coming s" />;
}
```

src/app/about/page.tsx:

```
import type { Metadata } from "next";
import { PageHeading } from "@components/page-heading";
export const metadata: Metadata = { title: "About" };
export default function AboutPage() {
  return <PageHeading title="About NurvexThink" subtitle="A software company that builds, publishes," />;
}
```

src/app/contact/page.tsx:

```
import type { Metadata } from "next";
import { PageHeading } from "@components/page-heading";
export const metadata: Metadata = { title: "Contact" };
export default function ContactPage() {
  return <PageHeading title="Contact" subtitle="Reach the NurvexThink team." />;
}
```

☐ Step 4: Verify all routes build

Run: `cd "d:/Company" && npm run build` Expected: route table lists `/`, `/products`, `/blog`, `/order`, `/about`, `/contact`; exit 0.

☐ Step 5: Commit

`git add -A && git commit -m "feat: add home hero and placeholder pages for all routes"`

Task 8: Wire Supabase clients (browser + server) and env example

Files: - Create: `src/lib/supabase/client.ts`, `src/lib/supabase/server.ts`, `src/lib/supabase/client.test.ts`
 - Create: `.env.example` - Modify: `src/app/layout.tsx` only if needed (no change expected)

Interfaces: - Consumes: env vars `NEXT_PUBLIC_SUPABASE_URL`, `NEXT_PUBLIC_SUPABASE_ANON_KEY`. - Produces: `createBrowserSupabaseClient()` (client components) and `createServerSupabaseClient()` (server components/route handlers), used by all data-access code in later plans.

☐ Step 1: Install Supabase libs

Run:

`cd "d:/Company" && npm install @supabase/supabase-js @supabase/ssr`

Expected: deps added.

☐ Step 2: Create `.env.example`

```
# Public (safe to expose; RLS protects data)
NEXT_PUBLIC_SUPABASE_URL=https://axbsghyqhhdaiylcksbv.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=

# Server-side ONLY – never expose to the browser, never commit a real value
SUPABASE_SERVICE_ROLE_KEY=
```

❑ Step 3: Write the failing test for the browser client factory

Create `src/lib/supabase/client.test.ts`:

```
import { describe, it, expect, beforeEach, vi } from "vitest";

describe("createBrowserSupabaseClient", () => {
  beforeEach(() => {
    vi.stubEnv("NEXT_PUBLIC_SUPABASE_URL", "https://axbsghyqhhdaiylcksbv.supabase.co");
    vi.stubEnv("NEXT_PUBLIC_SUPABASE_ANON_KEY", "test-anon-key");
  });

  it("returns a client exposing auth and from()", async () => {
    const { createBrowserSupabaseClient } = await import("@/lib/supabase/client");
    const client = createBrowserSupabaseClient();
    expect(client).toBeDefined();
    expect(typeof client.from).toBe("function");
    expect(client.auth).toBeDefined();
  });
});
```

❑ Step 4: Run test to verify it fails

Run: `cd "d:/Company" && npm test -- supabase` Expected: FAIL — cannot resolve `@/lib/supabase/client`.

❑ Step 5: Implement `src/lib/supabase/client.ts`

```
import { createBrowserClient } from "@supabase/ssr";

export function createBrowserSupabaseClient() {
  return createBrowserClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
  );
}
```

❑ Step 6: Implement `src/lib/supabase/server.ts`

```
import { createServerClient } from "@supabase/ssr";
import { cookies } from "next/headers";

export async function createServerSupabaseClient() {
  const cookieStore = await cookies();
  return createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll() {
          return cookieStore.getAll();
        },
        setAll(cookiesToSet) {
          try {

```

```

        cookiesToSet.forEach(({ name, value, options }) =>
            cookieStore.set(name, value, options),
        );
    } catch {
        // Called from a Server Component – safe to ignore; middleware refreshes the session.
    }
  },
},
},
);
}

```

☐ **Step 7: Run test to verify it passes**

Run: `cd "d:/Company" && npm test -- supabase Expected: PASS.`

☐ **Step 8: Verify typecheck + build**

Run: `cd "d:/Company" && npm run typecheck && npm run build Expected: exit 0.`

☐ **Step 9: Commit**

`git add -A && git commit -m "feat: add Supabase browser/server client factories and .env.example"`

Task 9: SEO metadata, favicon, and Open Graph from the logo

Files: - Create: `src/app/icon.png` (copied from brand logo), `src/app/opengraph-image.png` (copied from brand logo) - Modify: `src/app/layout.tsx` (metadataBase + OG metadata)

Interfaces: - Produces: site favicon + social card; no code interface.

☐ **Step 1: Copy the logo into app icon slots**

Run:

```

cd "d:/Company"
cp brand/NurvexThink_logo.jpeg src/app/icon.png
cp brand/NurvexThink_logo.jpeg src/app/opengraph-image.png

```

Expected: both files exist (Next derives favicon + OG image from these conventional filenames).

☐ **Step 2: Extend metadata in `src/app/layout.tsx`**

Update the metadata export:

```

export const metadata: Metadata = {
  metadataBase: new URL("https://nurvexthink.com"),
  title: { default: "NurvexThink", template: "%s – NurvexThink" },
  description: "Software, built and published. Custom software on demand.",
  openGraph: {
    title: "NurvexThink",
    description: "Software, built and published. Custom software on demand.",
    siteName: "NurvexThink",
    type: "website",
  },
  twitter: { card: "summary_large_image", title: "NurvexThink" },
};

```

☐ **Step 3: Verify build**

Run: `cd "d:/Company" && npm run build` Expected: exit 0; build output references /icon.png and /opengraph-image.png.

❑ **Step 4: Commit**

```
git add -A && git commit -m "feat: add favicon, OG image, and SEO metadata"
```

Task 10: GitHub Actions → Vercel deploy pipeline + CI checks

Files: - Create: `.github/workflows/ci.yml`, `.github/workflows/deploy.yml` - Create: `docs/DEPLOY.md`

Interfaces: - Produces: CI that runs lint/typecheck/test/build on PRs; deploy workflow that ships main to Vercel using repo secrets `VERCEL_TOKEN`, `VERCEL_ORG_ID`, `VERCEL_PROJECT_ID`.

❑ **Step 1: Create `.github/workflows/ci.yml`**

```
name: CI
on:
  pull_request:
    branches: [main]
  push:
    branches: [main]
jobs:
  verify:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: npm
      - run: npm ci
      - run: npm run lint
      - run: npm run typecheck
      - run: npm test
      - run: npm run build
    env:
      NEXT_PUBLIC_SUPABASE_URL: https://axbsghyqhhdaiylcksbv.supabase.co
      NEXT_PUBLIC_SUPABASE_ANON_KEY: build-time-placeholder
```

❑ **Step 2: Create `.github/workflows/deploy.yml`**

```
name: Deploy
on:
  push:
    branches: [main]
concurrency:
  group: production-deploy
  cancel-in-progress: true
jobs:
  deploy:
    runs-on: ubuntu-latest
    env:
      VERCEL_ORG_ID: ${ secrets.VERCEL_ORG_ID }
      VERCEL_PROJECT_ID: ${ secrets.VERCEL_PROJECT_ID }
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
```

```

with:
  node-version: 20
  cache: npm
- run: npm i -g vercel@latest
- run: vercel pull --yes --environment=production --token=${{ secrets.VERCEL_TOKEN }}
- run: vercel build --prod --token=${{ secrets.VERCEL_TOKEN }}
- run: vercel deploy --prebuilt --prod --token=${{ secrets.VERCEL_TOKEN }}

```

❑ Step 3: Create docs/DEPLOY.md

Deploy – NurvexThink Website

Deploys run via GitHub Actions (`.github/workflows/deploy.yml`) on every push to `main`, using the Vercel CLI + a token. This avoids Vercel Hobby's org-private-repo and commit-author restrictions, so all admins' merged work deploys for free.

One-time setup

1. Create the Vercel project (link it to this repo, or create empty and run `vercel link`).
2. In Vercel project settings, get `VERCEL_ORG_ID` and `VERCEL_PROJECT_ID` (from `.vercel/project.json` after `vercel link`, or the dashboard).
3. Create a Vercel access token (Account Settings → Tokens).
4. In the GitHub repo: Settings → Secrets and variables → Actions, add:
 - `VERCEL_TOKEN`, `VERCEL_ORG_ID`, `VERCEL_PROJECT_ID`
5. In Vercel project env vars, add `NEXT_PUBLIC_SUPABASE_URL`, `NEXT_PUBLIC_SUPABASE_ANON_KEY`, and (server-only) `SUPABASE_SERVICE_ROLE_KEY`.

Branch protection

Protect `main`: require a PR + 1 review; require the CI workflow to pass before merge.

❑ Step 4: Validate workflow YAML locally

Run:

```
cd "d:/Company" && node -e "const fs=require('fs');['ci','deploy'].forEach(f=>{const s=fs.readFileSync"
```

Expected: prints workflows ok.

❑ Step 5: Commit

```
git add -A && git commit -m "ci: add CI checks and GitHub Actions Vercel deploy pipeline"
```

Task 11: README and final verification

Files: - Create: README.md

Interfaces: - Produces: project README; no code interface.

❑ Step 1: Create README.md

NurvexThink – Main Website

The public hub for NurvexThink: product showcase, blog, custom-order intake, and an admin dashboard. Built with Next.js 15 + Supabase, deployed on Vercel.

Stack

TypeScript · Next.js 15 (App Router) · Tailwind CSS v4 · shadcn/ui · React Three Fiber (3D) · Supabase (Postgres/Auth/Storage) · Vercel.

Develop

```
```bash
cp .env.example .env.local # fill in the Supabase anon key
npm install
npm run dev # http://localhost:3000
```

## Scripts

npm run dev · npm run build · npm run lint · npm run typecheck · npm test · npm run format

## Docs

- Design spec: docs/superpowers/specs/2026-06-30-nurvexthink-website-design.md (+ PDF)
- Deploy: docs/DEPLOY.md
- Project conventions: CLAUDE.md

- [ ] **\*\*Step 2: Full verification pass\*\***

Run:

```
```bash
```

```
cd "d:/Company" && npm run format:check && npm run lint && npm run typecheck && npm test && npm run build
```

Expected: all green, build exit 0.

☐ Step 3: Commit

```
git add -A && git commit -m "docs: add project README"
```

Self-Review

Spec coverage (Foundation scope): - Stack (spec §3) → Tasks 1–8. ✓ - Brand/theme (spec §2, §7) → Task 4. ✓ - Layout + pages (spec §4) → Tasks 6–7 (placeholders; full content in later plans). ✓ - Supabase wiring (spec §3, §6) → Task 8; schema/RLS deferred to Backend plan. ✓ (noted) - Deployment + GitHub (spec §7) → Task 10. ✓ - Performance rules (spec §9) → enforced via skills + Server-Component-first structure. ✓ - 3D (spec §9), data model/RLS (spec §5), public content, admin (spec §4) → **deferred to later milestone plans** (intentional, stated up front). ✓

Placeholder scan: No “TBD/TODO” left as work items; “Coming soon” copy in page bodies is intentional placeholder *content*, replaced in later plans.

Type consistency: cn (Task 3) used in Task 6; NAV_LINKS typed in Task 6 Step 1 and consumed in Step 4; createBrowserSupabaseClient/createServerSupabaseClient (Task 8) match the test and the Interfaces blocks; PageHeading props match across Task 7 usages.

Note on tests: Pure config/scaffold tasks (1, 2, 5, 9, 10, 11) verify via build/lint/typecheck with expected output rather than unit tests, since there is no unit to assert. Logic units (cn, Navbar, Supabase factory) use real TDD.